

[과제1] OR 게이트 학습 및 시각화

```
[1] import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

✓ 과제1. OR 게이트 학습 및 시각화

```
[15] X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 1, 1, 1])
```

```
[19] model = Sequential()
model.add(Dense(4, input_dim=2, activation='relu')) #은닉층 1개
model.add(Dense(1, activation='sigmoid'))
```

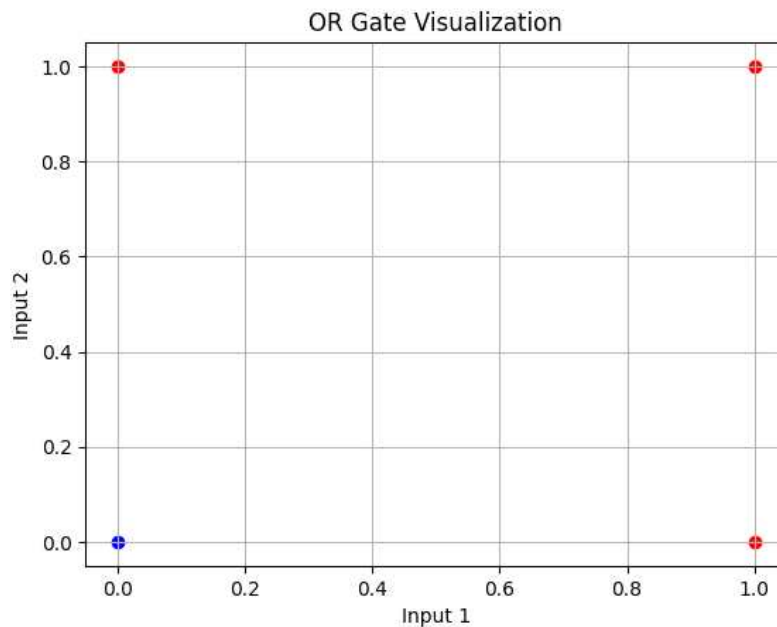
```
[22] model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(X, y, epochs=1000, verbose=0)
```

```
<keras.src.callbacks.history.History at 0x7e7c7a6449d0>
```

```
[23] pred = model.predict(X)
print("OR 예측 결과:\n", np.round(pred))
```

```
WARNING:tensorflow:5 out of the last 6 calls to <function TensorFlowTrainer.make_predict_
1/1 _____ 0s 71ms/step
OR 예측 결과:
[[0.]
 [1.]
 [1.]
 [1.]]
```

Dense나 epochs 수가 낮으면 첫 번째 값도 1로 예측하여 수를 상당 높임.



시각화에서 알 수 있듯이,

OR 게이트에서는 선형 분리가 가능하므로 은닉층 한 개로 충분.

[과제2] XOR 게이트 학습 및 시각화(ANN)

✓ 과제2. XOR 게이트 학습 및 시각화

```
[24] X = np.array([[0,0], [0,1], [1,0], [1,1]])
     y = np.array([[0], [1], [1], [0]])

[25] model2 = Sequential()
     model2.add(Dense(4, input_dim=2, activation='relu')) # 은닉층 1개
     model2.add(Dense(1, activation='sigmoid'))

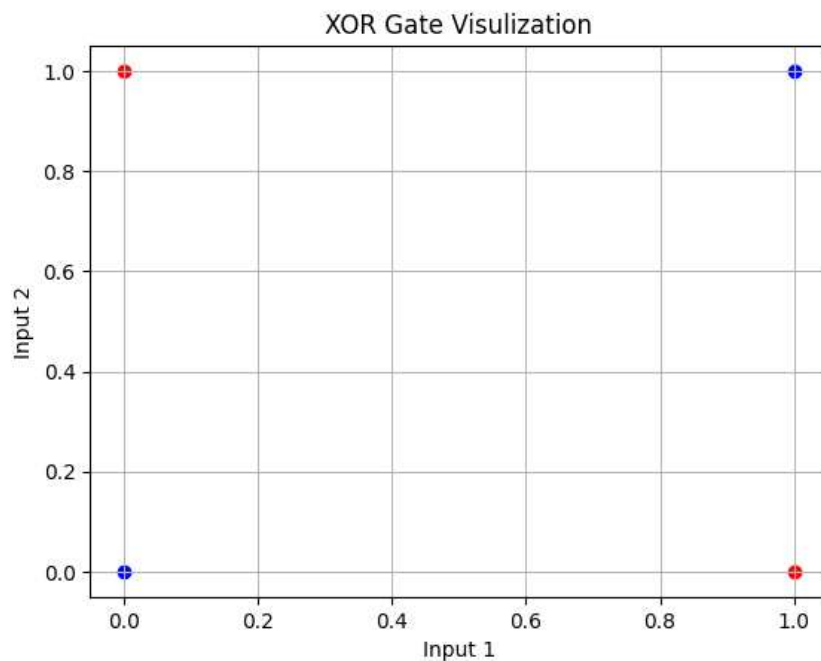
[26] model2.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
     model2.fit(X, y, epochs=1000, verbose=0)

<keras.src.callbacks.history.History at 0x7e7c7a1b7f90>

[27] pred = model2.predict(X)
     print("XOR 예측 결과:\n", np.round(pred))

WARNING:tensorflow:6 out of the last 7 calls to <function TensorFlowTrainer.make_pr
1/1 _____ 0s 83ms/step
XOR 예측 결과:
[[0.]
 [0.]
 [1.]
 [0.]
```

동일하게 진행한 결과, epochs 수를 높였음에도 틀린 답이 존재.



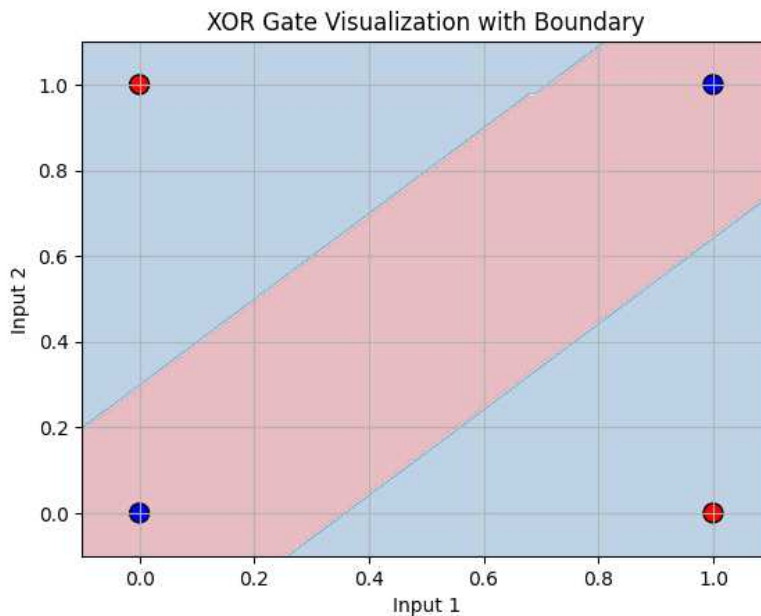
시각화에서 알 수 있듯이, 두 클래스가 교차하는 패턴을 보여 은닉층 1개로는 정확히 분리할 수 없어 학습 성능이 떨어짐.

[과제3] XOR 게이트 학습 및 시각화(DNN)

✓ 과제3. XOR 게이트 학습 및 시각화 (DNN)

```
[29] x = np.array([[0,0], [0,1], [1,0], [1,1]])  
     y = np.array([[0], [1], [1], [0]])  
  
[30] model3 = Sequential()  
     model3.add(Dense(4, input_dim=2, activation='relu')) # 첫 번째 은닉층  
     model3.add(Dense(4, activation='relu'))              # 두 번째 은닉층 2, 기존 1개에서 추가.  
     model3.add(Dense(1, activation='sigmoid'))  
  
[31] model3.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])  
     model3.fit(X, y, epochs=1000, verbose=0)  
  
<keras.src.callbacks.history.History at 0x7e7c72f0df10>  
  
[32] pred = model3.predict(X)  
     print("XOR 예측 결과 (DNN):", np.round(pred))  
  
1/1 ————— 0s 77ms/step  
XOR 예측 결과 (DNN):  
[[0.]  
 [1.]  
 [1.]  
 [0.]]
```

동일한 Dense와 epochs 수에서 은닉층을 하나 추가하였고, 예측이 알맞게 나오는 것을 확인.



다음과 같이 은닉층을 두 개 사용하므로 위와 같이 비선형 결정 경계를 생성해낼 수 있다.